

Manipal School of Information Sciences

Linux Operating System Theory Assignment

Bash Scripting: Arguments, File Operators and Filters

Five Scripts for File Checks and Log Investigation

Student Name:	Ribhav Devprasad
Reg. No.:	26110690002
Date:	3 November 2026

Bash Scripts: Arguments, File Operators, Read and Filters

Objective and setup

Five Bash scripts that take command-line arguments (`$1`, `$#`), test files with operators (`-e -f -d -r -w -x -s`) and filter text with `grep` and `head`. Each script validates its input before doing any work and exits with a non-zero status on error. Every script was run and the output below is the real output of that run.

Operator	True when
`-e`	path exists
`-f`	regular file
`-d`	directory
`-r` / `-w` / `-x`	readable / writable / executable by the user running the script
`-s`	file exists and has size greater than zero

Test files used:

```
$ ls -l sample.txt empty.txt mydir run_me.sh noread.txt server.log auth.log
-rw-r--r-- 1  761 Nov  4 04:08 auth.log
-rw-r--r-- 1    0 Nov  4 04:08 empty.txt
---------- 1    7 Nov  4 04:08 noread.txt
-rwxr-xr-x 1   23 Nov  4 04:08 run_me.sh
-rw-r--r-- 1  100 Nov  4 04:08 sample.txt
-rw-r--r-- 1  622 Nov  4 04:08 server.log

mydir:
total 0
```

Script 1: check_file.sh (argument, existence, head)

check_file.sh

```
#!/bin/bash
# check_file.sh - show the first 5 lines of a file given as the first argument

# 1. Was an argument supplied?
if [ $# -lt 1 ]; then
    echo "Error: no filename supplied." >&2
    echo "Usage: $0 <filename>" >&2
    exit 1
fi

FILE="$1"

# 2. Does it exist (as a regular file)?
if [ -f "$FILE" ]; then
    echo "File '$FILE' exists. First 5 lines:"
    head -n 5 "$FILE"
else
```

```
    echo "Error: '$FILE' does not exist or is not a regular file." >&2
    exit 2
fi
```

No argument

```
$ ./check_file.sh; echo "exit status: $?"
Error: no filename supplied.
Usage: ./check_file.sh <filename>
exit status: 1
```

File that exists (sample.txt has 7 lines; only 5 are shown)

```
$ ./check_file.sh sample.txt
File 'sample.txt' exists. First 5 lines:
line 1: alpha
line 2: bravo
line 3: charlie
line 4: delta
line 5: echo
```

File that does not exist

```
$ ./check_file.sh missing.txt; echo "exit status: $?"
Error: 'missing.txt' does not exist or is not a regular file.
exit status: 2
```

How it works. `$#` holds the number of arguments, so `-lt 1` detects a missing one. `-f` tests for an existing regular file. Errors go to standard error (`>&2`) and the exit status (1 or 2) tells callers what went wrong.

Script 2: perm_checker.sh (file type and permissions)

perm_checker.sh

```
#!/bin/bash
# perm_checker.sh - report the type and the current user's permissions on a path

if [ $# -lt 1 ]; then
    echo "Usage: $0 <path>" >&2
    exit 1
fi

P="$1"

if [ ! -e "$P" ]; then
    echo "Error: '$P' does not exist." >&2
    exit 2
fi

# Type
if [ -f "$P" ]; then
    echo "$P is a regular file."
elif [ -d "$P" ]; then
    echo "$P is a directory."
else
    echo "$P exists but is neither a regular file nor a directory."
fi

# Permissions for the user running the script
[ -r "$P" ] && echo "  Readable   : yes" || echo "  Readable   : no"
```

```
[ -w "$P" ] && echo "  Writable   : yes" || echo "  Writable   : no"
[ -x "$P" ] && echo "  Executable : yes" || echo "  Executable : no"
```

Regular file

```
$ ./perm_checker.sh sample.txt
sample.txt is a regular file.
  Readable   : yes
  Writable   : yes
  Executable : no
```

Directory

```
$ ./perm_checker.sh mydir
mydir is a directory.
  Readable   : yes
  Writable   : yes
  Executable : yes
```

Executable script

```
$ ./perm_checker.sh run_me.sh
run_me.sh is a regular file.
  Readable   : yes
  Writable   : yes
  Executable : yes
```

File with no permissions, checked as an unprivileged user (nobody)

```
$ ./perm_checker.sh noread.txt
noread.txt is a regular file.
  Readable   : no
  Writable   : no
  Executable : no
```

Non-existent path

```
$ ./perm_checker.sh /no/such/path; echo "exit status: $?"
Error: '/no/such/path' does not exist.
exit status: 2
```

> The root account bypasses read and write permission checks, so the no-permission test was run as the unprivileged user `nobody`. A normal user on your own machine will see the same result without needing this.

Script 3: log_search.sh (search a log for four keywords)

`log_search.sh`

```
#!/bin/bash
# log_search.sh - search a log file for failed, denied, unauthorized and error

if [ $# -lt 1 ]; then
    echo "Usage: $0 <logfile>" >&2
    exit 1
```

```
fi
LOG="$1"

if [ ! -f "$LOG" ] || [ ! -r "$LOG" ]; then
    echo "Error: '$LOG' is not a readable file." >&2
    exit 2
fi

for WORD in failed denied unauthorized error; do
    COUNT=$(grep -i -c -- "$WORD" "$LOG")
    echo "=== '$WORD': $COUNT matching line(s) ==="
    if [ "$COUNT" -gt 0 ]; then
        grep -i -n -- "$WORD" "$LOG"
    fi
    echo
done

TOTAL=$(grep -i -c -E 'failed|denied|unauthorized|error' "$LOG")
echo "Lines containing at least one keyword: $TOTAL"
```

Run on server.log

```
$ ./log_search.sh server.log
=== 'failed': 2 matching line(s) ===
2:Nov 03 10:02:45 srv sshd[402]: Failed password for root from 10.0.0.25
7:Nov 03 10:07:08 srv sshd[455]: Failed password for admin from 172.16.0.8

=== 'denied': 2 matching line(s) ===
5:Nov 03 10:05:37 srv web[517]: GET /etc/passwd 403 Access denied
10:Nov 03 10:10:02 srv kernel: Permission denied: /var/secure

=== 'unauthorized': 1 matching line(s) ===
4:Nov 03 10:04:19 srv web[517]: GET /admin 401 Unauthorized

=== 'error': 2 matching line(s) ===
3:Nov 03 10:03:02 srv app[311]: ERROR: database connection timed out
8:Nov 03 10:08:23 srv app[311]: error: disk quota exceeded

Lines containing at least one keyword: 7
```

Missing argument and missing file

```
$ ./log_search.sh; ./log_search.sh nofile.log; echo "exit status: $?"
Usage: ./log_search.sh <logfile>
Error: 'nofile.log' is not a readable file.
exit status: 2
```

How it works. A `for` loop runs `grep -i` (case-insensitive) for each keyword. `-c` counts matching lines, and `-n` prints the line numbers so an investigator can find each event. `--` stops `grep` treating a keyword that starts with `-` as an option. The final `grep -E 'a|b|c|d'` counts lines matching any keyword once, which is why the total can be lower than the sum of the four counts: a line such as *Permission denied* counts for one keyword only, but a line holding two keywords would be counted twice in the loop.

Script 4: file_status.sh (exists, regular, empty or non-empty)

file_status.sh

```
#!/bin/bash
# file_status.sh - check existence, regular-file status and emptiness

if [ $# -lt 1 ]; then
```

```
    echo "Usage: $0 <filename>" >&2
    exit 1
fi
F="$1"

if [ ! -e "$F" ]; then
    echo "'$F' does not exist."
    exit 2
fi
echo "'$F' exists."

if [ ! -f "$F" ]; then
    echo "'$F' is not a regular file."
    exit 3
fi
echo "'$F' is a regular file."

if [ -s "$F" ]; then
    echo "'$F' is non-empty. First 5 lines:"
    head -n 5 "$F"
else
    echo "'$F' is empty."
fi
```

Non-empty regular file

```
$ ./file_status.sh sample.txt
'sample.txt' exists.
'sample.txt' is a regular file.
'sample.txt' is non-empty. First 5 lines:
line 1: alpha
line 2: bravo
line 3: charlie
line 4: delta
line 5: echo
```

Empty regular file

```
$ ./file_status.sh empty.txt
'empty.txt' exists.
'empty.txt' is a regular file.
'empty.txt' is empty.
```

Directory

```
$ ./file_status.sh mydir; echo "exit status: $?"
'mydir' exists.
'mydir' is not a regular file.
exit status: 3
```

Missing file

```
$ ./file_status.sh nothing.txt; echo "exit status: $?"
'nothing.txt' does not exist.
exit status: 2
```

Script 5: investigate.sh (interactive investigation)

investigate.sh

```
#!/bin/bash
# investigate.sh - interactive log search for an investigator

if [ $# -lt 1 ]; then
    echo "Usage: $0 <logfile>" >&2
    exit 1
fi
LOG="$1"

# Verify the file exists and is readable
if [ ! -e "$LOG" ]; then
    echo "Error: '$LOG' does not exist." >&2
    exit 2
fi
if [ ! -r "$LOG" ]; then
    echo "Error: '$LOG' is not readable." >&2
    exit 3
fi

echo -n "Enter a search term: "
read -r TERM_IN

if [ -z "$TERM_IN" ]; then
    echo "Error: the search term cannot be empty." >&2
    exit 4
fi

TOTAL=$(grep -i -c -F -- "$TERM_IN" "$LOG")

if [ "$TOTAL" -eq 0 ]; then
    echo "No records match '$TERM_IN'."
    exit 0
fi

echo "First 5 matching records:"
grep -i -F -- "$TERM_IN" "$LOG" | head -n 5
echo "Total matching records: $TOTAL"
```

Search term with more than 5 matches (the term typed at the prompt is piped in here: `Nov`)

```
$ echo 'Nov' | ./investigate.sh server.log
Enter a search term: First 5 matching records:
Nov 03 10:01:11 srv app[311]: Service started
Nov 03 10:02:45 srv sshd[402]: Failed password for root from 10.0.0.25
Nov 03 10:03:02 srv app[311]: ERROR: database connection timed out
Nov 03 10:04:19 srv web[517]: GET /admin 401 Unauthorized
Nov 03 10:05:37 srv web[517]: GET /etc/passwd 403 Access denied
Total matching records: 10
```

Search term typed in a different case (matching is case-insensitive)

```
$ echo 'ERROR' | ./investigate.sh server.log
Enter a search term: First 5 matching records:
Nov 03 10:03:02 srv app[311]: ERROR: database connection timed out
Nov 03 10:08:23 srv app[311]: error: disk quota exceeded
Total matching records: 2
```

Search term with no match

```
$ echo 'ransomware' | ./investigate.sh server.log
Enter a search term: No records match 'ransomware'.
```

Empty search term

```
$ echo '' | ./investigate.sh server.log; echo "exit status: $?"
Enter a search term: Error: the search term cannot be empty.
exit status: 4
```

Unreadable file (as user nobody)

```
$ echo 'x' | ./investigate.sh noread.txt; echo "exit status: $?"
Error: 'noread.txt' is not readable.
exit status: 3
```

Missing file

```
$ echo 'x' | ./investigate.sh nofile.log; echo "exit status: $?"
Error: 'nofile.log' does not exist.
exit status: 2
```

How it works. The script first verifies the file with `-e` and `-r`, then prompts with `echo -n` and reads the term with `read -r`. `grep -F` treats the term as plain text so characters like `.` or `*` are not interpreted as regular-expression operators, which is the safer choice for IP addresses such as `10.0.0.25`. The first five matches come from `grep | head -n 5` and the total comes from `grep -c`.